Dexterous In-Hand Reorientation Using Reinforcement Learning

Pyae Sone Nyo Hmine Martin Peticco Alessandro Briseno-Tapia

***Abstract*—In this report, we study how reward shaping and curriculum design affect goal-conditioned in-hand manipulation with a low-cost anthropomorphic LEAP hand. We focus on a simple but challenging task: repeatedly reorienting a cube about its yaw axis on command, and then extending the same framework to full 3D pose control. Our aim is to document which practical design choices make long-horizon, goal-conditioned policies reliable on this platform, and how learning-based methods can be sensitive to being stuck in local minima.**

We begin from an existing 1D rotation setup that only learns to continuously spin the cube in a single direction, and iteratively modify the observation space, reward terms, and training curriculum. Key changes include adding explicit goal conditioning, a checkpoint-based curriculum over yaw, a streak-based bonus for consecutive successes, and finally providing the policy with pose observations and a capped rotation reward with near-goal angular velocity penalties. We evaluate four 1D variants in terms of success rate, time to goal, steady state error, and consecutive goals until failure, and then apply the final design to full 3D reorientation.

Our results show that the combination of pose feedback, conservative near goal shaping, and a curriculum that values consecutive successes is sufficient to obtain near-perfect 1D success rates and long success streaks, and produces stable, repeatable behavior that transfers cleanly to the 3D task.

I. Introduction

Dexterous manipulation is essential for human-like capabilities in robotics. The ability to operate in unstructured environments remains a key challenge, since in-hand reorientation of objects enables better autonomy and human-like interaction [1], [2]. Currently available open-source hands typically do not provide rich tactile or torque sensing capabilities, which are critical for stable manipulation [3].

Recent advances in reinforcement learning (RL) demonstrate promising capabilities for learning complex manipulation skills directly from interaction experience. Nevertheless, many existing approaches rely on hardware solutions or custom-built solutions [1], [4], [5]. These requirements restrict applicability and limit the deployment of dexterous manipulation in practical settings.

In this project, we investigate whether a simple, low-cost dexterous hand without rich sensing can learn goal-oriented in-hand object reorientation strictly using pose information. We propose a lightweight RL-based control framework that stabilizes and rotates an object. Through various simulation experiments, we evaluate its limits in one- and three-dimensional settings and demonstrate that meaningful dexterous manipulation is possible with such hardware.

II. Related Work

Various methods have been explored for learning-based in-hand manipulation, covering model-based, RL, and imitation-learning methods, and discussing different sensing, control, and hand architectures [6], [7]. These reviews highlight common tasks (reorientation, grasping, tool use), and tackle major challenges including high-dimensional control, contact modeling, sim-to-real transfer, and limited generalization across objects [6].

Seminal work demonstrates that RL can enable an anthropomorphic robotic hand (e.g. the high DoF "Shadow" hand) to manipulate and reorient objects using vision-based control and precise object pose tracking [1], [8]. More recent work builds on this foundation combining vision, tactile sensing, and proprioceptive state to execute complex manipulation tasks with a high success rate that generalizes across novel objects [4], [5], [9]. In one case, a system learns to reorient a cube relying only on tactile sensing [10]. Another example uses only "touch/no-touch" binary sensors with RL policies that transfer sim-to-real suggesting that low-cost hands and minimal sensing may suffice for certain dexterous tasks [11].

LEAP and other 3D-printed designs trade sensing fidelity for affordability making them appealing for wide deployment [19]. That said, the absence of tactile feedback significantly impairs the stabilization of objects during reorientation tasks [11]. Understanding the space of possible manipulation under these sensing constraints remains limited in the literature [12].

III. Approach

To run our model on various reorientation tasks, we use the LEAP hand, a low-cost, efficient, anthropomorphic hand that can be built for less than $2000 [19].

A. Simulation interface

The hand uses 16 independent ROBOTIS Dynamixel servo motors. For simulation, we rely on NVIDIA's Isaac Lab and the PhysX GPU-accelerated physics which enables rich manipulation dynamics and realistic modeling for both friction and compliance. We create an environment to batch many parallel simulations for massively increased RL throughput. We add features such as automatic resetting, observation stacking, and reward batching handled under the Isaac Lab framework. We configure each task through the Hydra-configurable tasks YAMLs, which allow for customizable physics fidelity, randomization settings, and more.

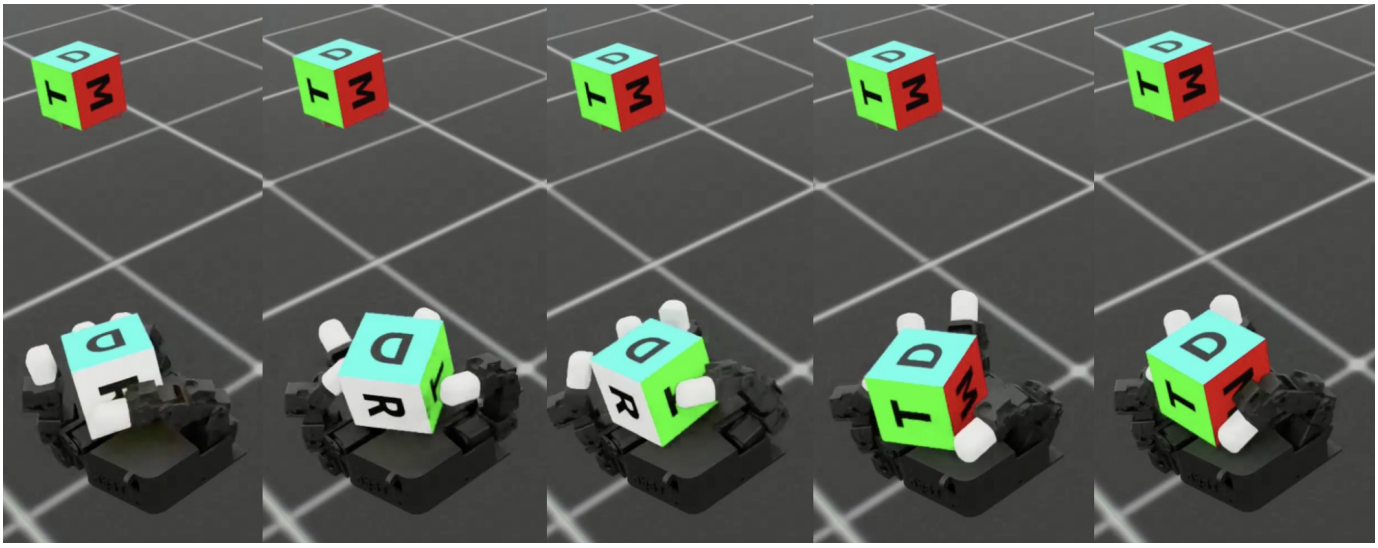

Fig. 1. A stop-motion viewe of our Version 4 1-dimensional in-hand cube reorientation policy reorienting a cube.

B. Reinforcement learning architecture

We use Proximal Policy Optimization (PPO) together with Generalized Advantage Estimation (GAE) and entropy regularization. The LEAP hand has internal motor readings, such as joint angle and velocity, for feedback. We train it on joint-space observations and the pose information. The actor and critic networks are both 3-layer MLPs, and then a Gated Recurrent Unit (GRU) gives the policy temporal memory.

IV. Experiments

A. Task setup & goal

The environment begins with the LEAP hand at the world origin, palm facing normal to the Z axis, and in its default joint arrangement with the fingers pointing outward. The cube spawns at the center of the palm when the episode start, with its size constant across experiments. The policy in training is run at 30Hz, but the environment's physics run at 120Hz.

B. Iterations of rewards and training curricula

The creators of the LEAP hand provide example code to train a policy which rotates a cube infinitely about the Z axis. However, this approach has several issues which make it incapable of achieving our goals. First, the policy is only conditioned on the position targets and joint angles of the 16 actuators from the previous 2 time steps. Second, the training curriculum involves giving a reward for every $+22.5^\circ$ positive increment. This means the policy only learns to spin the cube in one direction without a specific target angle. As a first pass at goal "conditioning", we first modify the environment to accept input from the user and pause the policy actions at the goal, implemented as a layer of code atop the existing policy rather than re-training it to achieve the goal angle. As a consequence of the 1-directional policy, a small negative change in the goal angle means a nearly 360º rotation.

To achieve true 1D control, we iterate through four versions of our reward function and training curricula.

1) 1D Version 1: To add goal conditioning, we pass the policy our current and target joint positions alongside the current orientation of the cube. We choose quaternions because they provide a compact 4-element representation of 3D goal orientation while avoiding issues such as gimbal lock (as with Euler angles), singularities (as with axis-angle), and excessive values (as with 9-element rotation matrices). Passing a simple 1D angle could also work, however, we want to more efficiently generalize to 3D. We continue to provide a history of the previous 2 time steps.

Our training curriculum is as follows:

1) Initialize the cube at a default position and uniformly sample a random goal angle between -180º and +180º.
2) Upon reaching that angle within a specified tolerance, provide a large reward.
3) Keep the reward constant for a random hold period of between 10 and 30 time steps.
4) Sample a new goal orientation between -180º and +180º.
5) Repeat steps 2-4 until failure or timeout.

The intuitive logic is to teach the policy how to achieve a goal angle, then pause to wait for a new target. Two failure modes can occur: (1) Falling, measured by exceeding a euclidean distance from the palm's center, or (2) Flipping, where the cube inverts, measured by calculating the dot product with the Z axis and maintaining this value above 0.5. For this first iteration, the reward terms include: a positional term penalizing the distance between the cube and a predefined center position, a rotational term that grows with the quaternion distance between the cube and the target orientation, an action regularization term based on the squared norm of the actions, a posture term penalizing deviation from a comfortable reference joint configuration, a torque term penalizing the squared joint torques, an angular velocity term that penalizes the cube's rotational speed, and a fingertip distance penalty to encourage continuous grasping of the cube. We also add a one-time

success bonus when the cube meets the goal and a fall penalty when it leaves the in-hand region.

2) 1D Version 2: In the second version, we keep the basic observation and reward structure from Version 1, but most notably, we add a checkpoint-based curriculum to help the policy progress toward the goal. The logic during training is as follows:

1) At the start of the episode, measure the distance between the cube's current yaw angle and the sampled goal yaw.
2) Create a set of intermediate checkpoints in $22.5°$ increments.
3) During training, every time the cube passes a checkpoint for the first time, we provide a bonus reward of 50 on top of the previous dense reward terms from Version 1. We only provide the reward the first time so it does not learn to oscillate around a checkpoint and only reward reaching the checkpoints in the correct order so it does not learn to take the long way around.
4) Upon reaching the goal angle, an even larger bonus is provided.

The multi goal and hold logic remain the same as in Version 1.

3) 1D Version 3: In Version 3, our main focus is improving consecutive success rates and drifting. We notice that after a few consecutive goals, the Version 2 policy loses track of the cube which starts to drift away from the palm, eventually falling off. To address this, we increase emphasis on consecutive successes, most notably by providing progressively larger rewards for each consecutive success. This way, the policy learns not to lose track of the cube.

Another change from Version 2 stems from concern that our penalty on angular velocity is hindering the policy's learning, as it has to accumulate many penalties to reach the checkpoint successes. To address this, we restrict the angular velocity penalty to only apply once the rotation error $|\theta_{\text{err}}|$ falls below a fixed threshold of $\approx$ 17 degrees. Outside this band, the policy is free to move quickly without punishment for rotational speed. Inside this band, the combination of rotation reward, angular velocity penalty, and checkpoint, goal bonuses encourages the cube to settle into a stable, low velocity configuration at the desired Z angle. Finally, we remove the penalty for being far from the hand home position since it may conflict with the hand's ability to learn the task. The checkpoint logic, success detection, multi goal structure, and underlying observation space remain as in Version 2.

4) 1D Version 4: Version 4 is our final 1D setup focused on providing full knowledge of the cube's current state. We extend observation to include the cube's current global position in addition to the previous joint positions, current position targets, cube orientation quaternion, and goal quaternion. The intention is to address the drifting issues that are observed in Version 3.

For rewards, we keep most of the same terms as Version 3, including distance, rotation, action regularization, posture penalty, torque penalty, and a near goal angular velocity penalty. The main change is that we cap the rotation reward

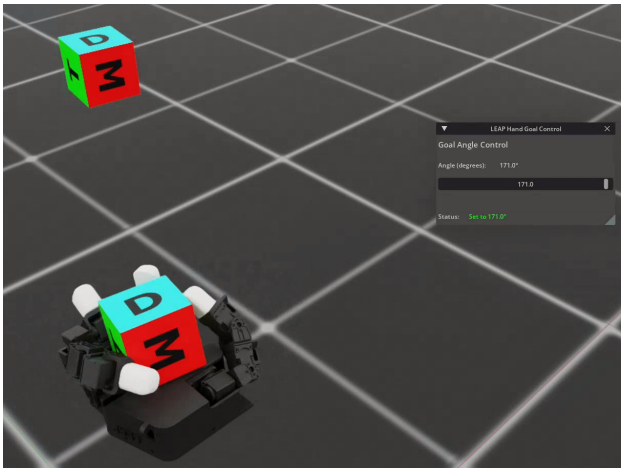

Fig. 2. Example of our Version 4 1D policy.

once the orientation error falls within the success tolerance, as we want to reduce twitching at the goal. Concretely, we first compute a term of the form $r_{\text{rot, raw}} = \frac{1}{|\theta_{\text{err}}|+\epsilon}$, then replace it by a constant value whenever $|\theta_{\text{err}}| \leq$ success_tolerance. This removes the incentive for the policy to "micro adjust" orientation by twitching around the goal to squeeze out extra reward. Together with the near goal angular velocity penalty, this encourages the policy to rotate quickly when far away, but stop and hold once sufficiently close, instead of wiggling around the goal orientation. The checkpoint scheme and multi goal hold curriculum from Version 2 are left unchanged and still provide intermediate bonuses along the shortest direction to each new yaw target. Finally, we remove the fingertip distance penalty, as it gives the policy trouble learning the task. For our later extension to 3D reorientation, the policy should not need to explicitly be told to hold the cube still, as it will learn to maintain orientation naturally as part of the task. An example of our Version 4 policy running 1D reorientation is shown in Figure 2 and in stop-motion form in Figure 1.

5) Extension to full 3D reorientation: Separately from the 1D experiments, we generalize the task to full 3D object reorientation. The observation structure mirrors Version 4 of the 1D setup; for each timestep, the policy receives normalized hand joint positions, current joint position targets, the cube position relative to the environment origin, the cube orientation as a unit quaternion, and the desired goal orientation as a unit quaternion. We also continue to provide the policy a history of the two previous time steps.

The main change is in how we generate goals and measure pose error. Instead of sampling a scalar yaw angle on the Z axis, we now sample random target orientations over SO(3) and represent them directly as goal quaternions. The rotation error is computed as the geodesic distance between the current and goal quaternions, using the angle of the relative quaternion. Success is defined by combining three conditions: the geodesic rotation error must be below a tolerance, the cube must be within a small positional radius of the in-hand reference point, and the angular speed must be below a

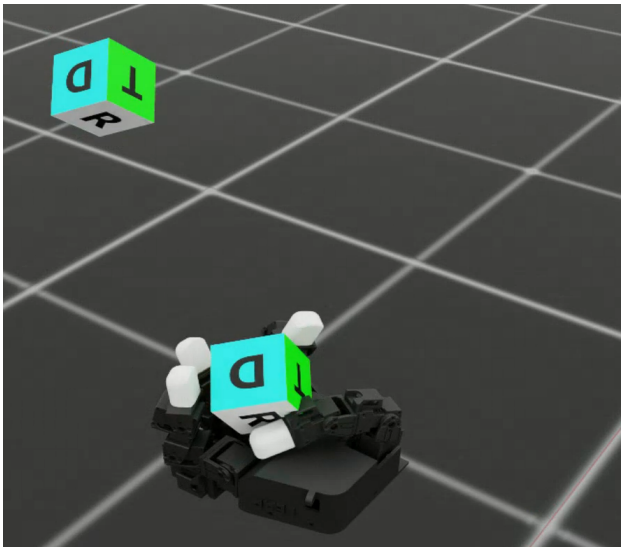

Fig. 3. Example of our 3D policy.

fixed threshold. Unlike the 1D case, we do not use a flipping condition in 3D, since any cube orientation is a valid target.

We reuse the same dense shaping terms as in the final 1D version: capped rotation reward, action, posture, torque, and a near goal angular velocity penalty. To extend our checkpoint curriculum to 3D, instead of measuring the Z angle delta, we measure geodesic rotation distance. The logic is the same, with a larger bonus at the final goal orientation. The multi goal logic is also the same as the 1D case. An example of our 3D policy running full 3D reorientation is shown in Figure 3.

V. Evaluation & Discussion

A. Evaluation protocol

To evaluate the 1D policies, we create 256 parallel environments for each version of the 1D task and repeatedly sample random goal orientations in each environment. The policy gets up to 10 seconds to achieve the goal. If it misses, a failure is recorded, and the environment is terminated. If it achieves the goal within a tolerance of 0.3 rad (as in the training), a success is recorded, and the policy is given another goal. This repeats until failure or until a global timeout of 64 seconds. We record the following metrics:

1) **Success rate**: The number of successful goal completions divided by the total number of goals given.
2) **Time to goal**: The time taken between the assignment of the goal and the achievement of the goal.
3) **Steady state error**: After a success, a 0.5 second holding period is recorded, and we record the average steady state error during that period.
4) **Goals until failure**: The number of consecutive successes until failure or the global timeout is reached.

B. Comparison between 1D policies

We will quantitatively compare the policies in terms of the above metrics. Results are summarized in Table I.

TABLE I
Summary of Evaluation Metrics for 1D Reorientation Variants

Metric	1Dbi	1Dbi2	1Dbi3	1Dbi4
Success rate	0.38	0.29	0.56	0.99
Time to goal mean (s)	1.0	0.62	1.2	0.70
Steady error mean (rad)	0.14	0.12	0.14	0.16
Goals until failure mean	0.61	0.42	1.3	23

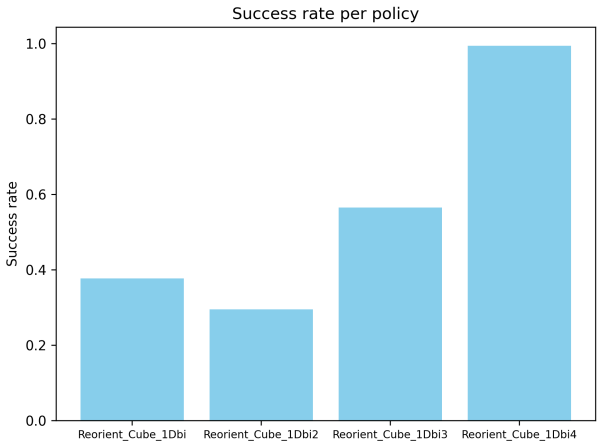

Fig. 4. Comparison of success rates across 1D policies.

1) Success Rate: The manner in which we shape our training curricula greatly influences the success rates of our policies as seen in Figure 4. The biggest difficulty is the fact that our Versions 1-3 policies do not have access to the true position of the cube, only the orientation. The Version 1 policy only succeeds in about 38% of commanded goals which matches our qualitative impression that it can make progress toward the goal at first but often loses track of the cube quickly. We introduce the checkpoint curriculum in 1Dbi2 to address this and make the reward function less sparse, giving the policy a strong incentive to march through intermediate yaw angles toward the final goal angle. We believe this encourages fast, single-pass trajectories, but also biases the learning signal toward being content with only hitting a few checkpoints, resulting in an even lower success rate of 29%.

In Version 3, we begin to progressively increase the bonus reward for each consecutive success to encourage reaching the final goal instead of just checkpoints. This increases the success rate to 56% without changing the basic observation space. In Version 4, once we add explicit cube position, the policy can adjust its movement in real-time to ensure that the cube is on track to reaching the goal position. This is reflected in its near-perfect 0.99 success rate. This behavior is consistent even when maximum episode length is extended to 128 seconds.

2) Time to Goal: The time to goal trends seen in Figure 5 tend to correspond with the amount we penalize or encourage motion in each version. Version 1 has fairly strong penalties on angular velocity throughout the trajectory, along with several regularization terms, which result in the policy being more

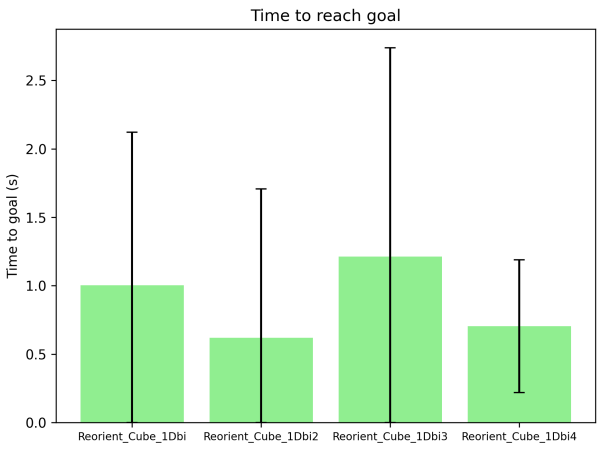

Fig. 5. Average time taken for 1D policies to achieve commanded goal angle.

conservative. It goes to the goal, but tends to be actively penalized for arriving quickly. The checkpoint-based curriculum introduced in Version 2 provides the policy a more direct path in reward space, encouraging it to reach goals faster, which results in a lower average time for Version 2.

In Version 3, we relax several penalties away from the goal and only keep a strong angular velocity cost near the target band, which lets the hand spin the cube more freely before it settles. At the same time, we add an increased distance penalty and push more for consecutive successes, which likely results in the policy being more cautious on average. This arises as a longer time to goal than Version 2. Version 4 explicitly addresses this issue by providing access to position information to allow for online correction and a capped rotation reward near the target. These make the policy free to move quickly when rotation error is large, and once it approaches the goal, it has both the information and the incentive to stop. The result is that Version 4 is nearly the fastest policy despite having a significantly higher success rate.

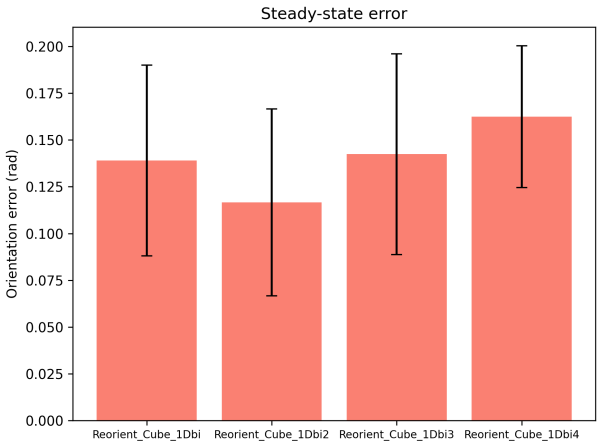

Fig. 6. Mean error across 0.5s holding period across 1D policies.

3) Steady state error: The steady state error seen in Figure 6 aligns with the tradeoff between solid at-goal behavior with repeatability for each version. In Version 1 and especially Version 2, we use a fairly heavy mix of posture regularization, fingertip distance penalties, and a global angular velocity penalty. Once the cube approaches the goal, these terms push the hand into a tight, constrained configuration that attempts to freeze the cube in place. That gives Version 2 the lowest steady state error of the four, but maintaining position accuracy falters, resulting in low repeatability. The fact that Version 3 has similar steady state error while relaxing some of these costs fits this picture. Even without pose, so long as the policy penalizes motion near the goal, it will settle into a small error band for the successful episodes we measure.

Version 4's pose information and capping of the rotation reward inside the success tolerance implies that there is no longer an incentive to make corrections once the error is "good enough," which this is done with the intention of reducing shaking. We also remove the fingertip distance term that formerly squeezed the hand tightly around the cube, as it slows the policy down. Combined with the near goal angular velocity penalty, the easiest way for the policy to obtain high return is to drive the rotation error into the tolerance band, then stop moving, rather than arrive at the exact angle. That appears as a slightly higher mean steady state error, but it is still well below the 0.3 rad success threshold and matches what we see qualitatively. Version 4 goes to the right place, holds without obvious twitching, and, unlike the earlier versions, keeps doing so across long sequences of goals to collect the progressively higher consecutive rewards.

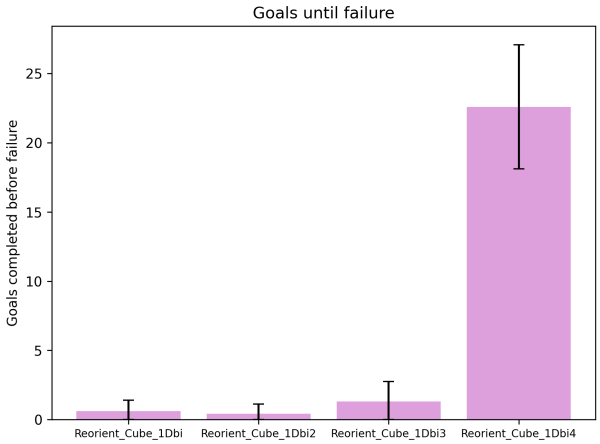

Fig. 7. Average number of consecutive successful goals until failure or timeout.

4) Goals until failure: Versions 1 and 2 are designed around single goal success with some holding, and the reward structure reflects that. The hand is rewarded once for reaching a target and then given a short hold period, but there is no explicit notion that staying reliable over many goals is more valuable. Combined with the lack of pose information, this leads to the policy working for a few goals, but drifting and

failing the next goal, which is why the mean number of goals until failure stays close to one.

To resolve this, Version 3 further emphasizes consecutive successes. We try reducing drift by adding an explicit centering term which uses a Gaussian reward to keep the cube near the center of the palm instead of just staying above the fall threshold.

$$r_{\text{center}} = \exp\left(-\frac{\|x_{\text{cube}} - x_{\text{ref}}\|}{\sigma_{\text{center}}}\right) \cdot \text{scale},$$

However, this causes issues with training down the line as sometimes the hand needs to move the cube away from the center to reach certain yaw angles. For this reason, we pivot to scaling the success bonus with the length of the current number of consecutive successes. This, along with the strengthened distance penalty, is why the policy can learn that reaching a few rewards and losing the cube is not as good as maintaining reliability.

The most significant change comes with Version 4, which provides missing state information. With access to cube pose, the policy can correct small drifts to achieve consecutive successes instead of relying on a fixed centering objective, and the capped reward near the goal removes the incentive to jitter for extra return. Once the cube has a good configuration, the safest and most rewarding strategy is simply to keep it there, which is exactly what produces the large jump in goals until failure for Version 4, as seen in Figure 7.

C. 3D reorientation

Despite our project's focus on 1D rotation, we attempt to extend our Version 4 policy to full 3D reorientation with some success. Qualitatively, the policy is able to achieve goal orientation in full 3D and keep it there. However, the policy exhibit a strong dichotomy between being able to achieve some goals reasonably well and barely getting close to achieving other goals. We believe this is because the shortest geodesic distance is not the optimal path to reorienting an object. One might imagine that it could be easier to rotate a cube one face at a time rather than rotating about its corner. This makes sense for 1D, as every configuration of a cube through a 1D rotation is stable, as it simply rests on the palm.

VI. Conclusion

We managde to extend a rudimentary example policy of continuous, one-dimensional rotation by training the LEAP hand with reinforcement learning. First, we achieved monodirectional control, then bidirectional control, and finally, a first pass at full 3D reorientation. Nevertheless, there are still several improvements to be made and potential iterations to continue this work.

Across these stages, our main findings are that relatively small but targeted changes to the observation space and reward shaping can make the difference between a policy that merely spins an object and one that reliably tracks commanded goals over long horizons. Adding explicit goal conditioning, a checkpoint style curriculum, and later pose information gives us stable, repeatable 1D performance and a workable starting point for 3D. The significance here is not in proposing a new algorithm, but in providing a concrete recipe and a set of failure cases that others can reuse when building goal-conditioned in-hand manipulation on similar low-cost hands. At the same time, our design remains tuned to a single hand–object setup and depends on carefully balanced penalties, so its generality and sample efficiency are still limited. The results should be viewed as a baseline and diagnostic tool rather than a final solution.

One key issue to address is the goal conditioning to train the hand to always take the most efficient path to the goal. The first solution is our bidirectional spin policy, but our implementation, while partially successful, presents another problem: the hand treats ±180º as a virtual wall that it cannot pass. For example, traveling from -179º to +179º should only warrant a delta of 2º in the most efficient case, but instead of viewing the angle space as a continuous circle, its virtual topology is akin to a line segment whose ends cannot be traversed. Instead, the hand travels 358º to arrive almost exactly where it started, albeit smoothly and stably. Rotating in the reverse direction is a significant improvement to our first controlled angle policy in 1-dimension, but given that there always exists a path $\leq$ 180º, the policy can still be optimized to always choose this direction.

There is still significant work to be done to achieve robust 3D reorientation. We predict it will mostly consist of repeated iteration on reward shaping until the balance of penalties is correct. Although the current error metric is geodesic quaternion distance, this may not be the most optimal. Axis-angle and Euler angle methods do exist and are worth exploring, or a new method specific to a the object geometry could be implemented. Stochastic methods for optimal trajectory planning or a separate set of parameters optimized to determine the ideal rotation set from a start to a goal pose could be computationally intense, but more efficient for training and deployment.

Acknowledgments

We would like to thank our TA, Bernhard Paus Græsdal, for his encouragement, interest in our project, and guidance throughout the checkoffs.

We would like to thank the Robotic Manipulation course staff, including Professor Tomás Lozano Pérez and the TAs, for their dedication to improving the class, answering our questions, and the learning of 6.4210/6.4212 students.

We would like to thank the Improbable AI Lab for access to the machine on which we trained and deployed our models.

References

[1] M. Andrychowicz et al., "Learning Dexterous In-Hand Manipulation," arXiv preprint arXiv:1808.00177, 2018.

[2] C. Yu and P. Wang, "Dexterous Manipulation for Multi-Fingered Robotic Hands With Reinforcement Learning: A Review," Frontiers in Neurorobotics, vol. 16, 2022.

[3] A. I. Weinberg et al., "Survey of learning-based approaches for robotic in-hand manipulation," Frontiers in Robotics and AI, 2024.

[4] H. Qi et al., “General In-Hand Object Rotation with Vision and Touch,” Proceedings of Machine Learning Research (PMLR), 2023.
[5] H. Yin, B. Huang, Y. Qin, Q. Chen and X. Wang, “Rotating without Seeing: Towards In-hand Dexterity through Touch,” Robotics: Science and Systems (RSS), 2023.
[6] A. I. Weinberg, A. Shirizly, O. Azulay, and A. Sintov, “Survey of Learning-Based Approaches for Robotic In-Hand Manipulation,” arXiv:2401.07915, 2024.
[7] C. Yu and P. Wang, “Dexterous Manipulation for Multi-Fingered Robotic Hands With Reinforcement Learning: A Review,” Frontiers in Neurorobotics, vol. 16, 2022.
[8] O. Andrychowicz *et al.*, “Learning Dexterous In-Hand Manipulation,” The International Journal of Robotics Research, 2020.
[9] G. Khandate, C. Mehlman, X. Wei and M. Ciocarlie, “Dexterous In-hand Manipulation by Guiding Exploration with Simple Sub-skill Controllers,” 2023.
[10] J. Pitz, L. Röstel, L. Sievers, and B. Bäuml, “Dextrous Tactile In-Hand Manipulation Using a Modular Reinforcement Learning Architecture,” arXiv:2303.04705, 2023.
[11] Z.-H. Yin, B. Huang, Y. Qin, Q. Chen, and X. Wang, “Rotating without Seeing: Towards In-hand Dexterity through Touch,” arXiv:2303.10880, 2023.
[12] W. Hu *et al.*, “Dexterous In-Hand Manipulation of Slender Cylindrical Objects Using Deep Reinforcement Learning with Tactile Feedback,” 2025.
[13] J. Si, K. Zhang, Z. Temel, O. Kroemer, “Tilde: Teleoperation for Dexterous In-Hand Manipulation Learning with a DeltaHand,” Robotics: Science and Systems (RSS) 2024.
[14] W. Qi *et al.*, “Human-like Dexterous Grasping Through Reinforcement Learning and Multimodal Perception,” 2025.
[15] X. Song *et al.*, “An overview of learning-based dexterous grasping: recent advances and future directions,” 2025.
[16] Shadow Robot Company, “The Shadow Dexterous Hand: Design Overview,” 2020.
[17] K. Adachi, T. Wada, and M. Inaba, “Allegro Hand: A Lightweight and Low-Cost Dexterous Hand for Human-like Manipulation,” IEEE-RAS Humanoids Workshop, 2014.
[18] R. Deimel and O. Brock, “A Compliant Hand based on a Novel Pneumatic Actuator,” IEEE International Conference on Robotics and Automation (ICRA), 2013.
[19] K. Shaw, A. Agarwal, and D. Pathak, “LEAP Hand: Low-Cost, Efficient, and Anthropomorphic Hand for Robot Learning,” arXiv:2309.06440 [cs.RO], 2023.